

Antigravity Final MVP Plan

Local AI Document Organization System

Project Goal

Build a simple internal web application for up to 5 users that automatically organizes legal/customer documents. Core flow: New/Existing Customer → Upload → Save original → OCR → Qwen3 1.7B → Name + document type → Customer matching/creation → Organization → Confirmation.

Technology

- Go backend
- Alpine.js + Tailwind CSS frontend
- PostgreSQL
- PaddleOCR PP-OCRv5 Mobile
- Qwen3 1.7B via Ollama
- Gemma 3 4B Vision via Ollama as a fallback
- Local filesystem storage
- Docker for reproducible setup if useful
- No Kubernetes, Kafka, microservices, Redis, cloud storage, or unnecessary orchestration

Phase 1: Foundation

- Set up Go, PostgreSQL, Alpine.js and Tailwind.
- Create users, customers and documents models.
- Create dashboard, navigation, upload and customer pages.
- Implement simple authentication for up to 5 users.
- Implement local storage using unique IDs.
- Do not implement AI yet.
- STOP, test, review and push to GitHub.

Phase 2: Upload & Customer Flow

- Implement New Customer / Existing Customer selection.
- Existing Customer gets a searchable customer selector.
- New Customer requires only the upload.
- Validate PDF, JPG, JPEG and PNG.
- Save the original file immediately.
- STOP, test and push to GitHub.

Phase 3: OCR

- Integrate PaddleOCR PP-OCRv5 Mobile.

- Support scanned PDFs, images and multi-page PDFs.
- Store OCR output without replacing the original.
- Add upload/processing/OCR status handling.
- STOP, test with real documents and push to GitHub.

Phase 4: Local AI Classification

- Integrate Ollama + Qwen3 1.7B.
- Extract person_name, document_type and confidence as JSON.
- Initial types: PAN, Aadhaar, Passport, Driving License, Property, Rent Agreement, Court/Legal, Bank/Financial, Insurance, Other.
- Low-confidence results require review.
- Do not fine-tune initially.
- STOP, test real documents and push to GitHub.

Phase 5: Customer Organization

- For new customers, extract the name and create a unique customer ID when no match exists.
- For existing customers, associate the document with the selected customer.
- Never use names as unique identifiers.
- If multiple customers have the same name, require confirmation.
- Organize documents by customer ID and document type.
- STOP, test and push to GitHub.

Phase 6: Vision Fallback

- Only add after the primary pipeline works.
- Use Gemma 3 4B Vision when OCR fails or produces insufficient text.
- Target poor scans, photographs and unusual layouts.
- Do not run vision for every document.
- Keep it only if testing shows a real improvement.
- STOP, test and push to GitHub.

Phase 7: UI/UX Integration

- Implement the uploaded UI using Alpine.js + Tailwind.
- Show: Uploading → Reading → Identifying → Finding customer → Organizing → Review → Completed.
- Show detected person, document type and confidence.
- Provide Confirm and Edit actions.
- STOP for UI review and GitHub push.

Phase 8: Final MVP Testing

- Test new/existing customers, duplicate names, multiple document types, poor scans, unsupported files, OCR/AI failures, low confidence, multiple uploads and authentication.

- Verify originals are never lost and IDs never collide.
- Verify no external AI API is called.
- Verify multiple users and the target server.

Development Rules

- Keep the MVP simple and avoid over-engineering.
- No microservices, Kubernetes, Kafka, Redis or unnecessary infrastructure.
- Do not fine-tune initially.
- Keep AI providers replaceable.
- Keep original documents untouched.
- Prefer local processing.
- Preserve the original file if processing fails.
- After every phase, STOP and wait for the user to test and push to GitHub.

Final Architecture

Browser → Alpine.js/Tailwind → Go → PostgreSQL + local storage → PaddleOCR → Qwen3 1.7B → Customer + document type → User Review → Organized document. Gemma 3 4B Vision is used only as an OCR fallback.

Definition of Done

Upload → Read → Identify person → Identify document → Find/create customer → Organize → Confirm.